

Programación Científica · 2026-1

Repaso: EDOs, el chisme y POO

Módulo 4 – Sesión integradora

De la ecuación al código, y del código a la clase

Universidad Nacional de Colombia

`mbastidaso@unal.edu.co`

Parte 1

Repaso: EDOs y sistemas dinámicos

Recordatorio rápido

Lo que ya vimos

Un sistema dinámico describe cómo cambia un **estado** $x(t)$ en el tiempo mediante una EDO:

$$\dot{x}(t) = f(x(t), t), \quad x(t_0) = x_0.$$

Si f es Lipschitz, Picard-Lindelöf garantiza solución única.

Hoy el estado es distinto

En vez de una opinión continua (H-K), cada persona está en uno de **tres estados discretos** frente a un chisme: no lo sabe, lo cuenta, o ya se cansó de contarlo.

El chisme como sistema dinámico

Tres estados (fracciones de la población):

- **Ignorante** (i): no conoce el chisme.
- **Difusor** (s): lo conoce y lo cuenta.
- **Cansado** (r): lo conoce pero ya no lo cuenta.

Reglas (Maki-Thompson):

- Ignorante + difusor $\rightarrow$ dos difusores (tasa β).
- Difusor + alguien informado $\rightarrow$ un cansado más (tasa α): se cansa al notar que ya lo sabían.

Modelo

$$\dot{i} = -\beta is, \quad \dot{s} = \beta is - \alpha s(s + r), \quad \dot{r} = \alpha s(s + r), \quad i + s + r = 1.$$

Repaso: Euler y RK4

Euler (orden 1)

$$w_0 = x_0,$$

$$w_{i+1} = w_i + h f(t_i, w_i)$$

Error global $O(h)$.

RK4 (orden 4)

$$w_0 = x_0,$$

$$k_1 = h f(t_i, w_i),$$

$$k_2 = h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_1\right),$$

$$k_3 = h f\left(t_i + \frac{h}{2}, w_i + \frac{1}{2}k_2\right),$$

$$k_4 = h f(t_i + h, w_i + k_3),$$

$$w_{i+1} = w_i + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4),$$

Error global $O(h^4)$.

Parte 2

De funciones a clases

El patrón que se repite

En el notebook, euler y rk4 son funciones sueltas que reciben siempre lo mismo:

```
def euler(f, x0, t, **kw): ...  
def rk4(f, x0, t, **kw): ...
```

Idea

Si el campo `f`, el estado y el paso `dt` siempre viajan juntos, agrupémoslos en un objeto que **recuerda su propio estado** entre llamadas.

Ingredientes mínimos de una clase

Vocabulario

- **class**: define un nuevo tipo.
- **__init__**: se ejecuta al crear el objeto.
- **self**: el objeto mismo.
- **atributo**: dato que vive en el objeto.
- **método**: función que vive en el objeto.

```
class Particula:
    def __init__(self, x, v):
        self.x = x
        self.v = v

    def mover(self, dt):
        self.x += self.v * dt

p = Particula(x=0.0, v=2.0)
p.mover(dt=0.5)
print(p.x)      # 1.0
```


La clase Solver y su subclase Euler

```
class Solver:
    def __init__(self, f, x0, dt, **kwargs):
        self.f, self.dt, self.kwargs = f, dt, kwargs
        self.x = np.array(x0, dtype=float)
        self.t = 0.0

    def step(self):
        raise NotImplementedError(
            "Las subclases deben implementar step()")

class Euler(Solver):
    def step(self):
        self.x = self.x + self.dt * self.f(
            self.x, self.t, **self.kwargs)
        self.t += self.dt
```

Herencia

Euler hereda `__init__` sin reescribirlo y solo sobrescribe `step()`.

Ejercicio: complete RK4(Solver)

En el notebook

Complete `step()` de `RK4(Solver)` con la fórmula de RK4, reutilizando `self.f`, `self.x`, `self.t`, `self.dt`, `self.kwargs`. Verifique que coincide con `rk4()` aplicada al mismo `dk_rhs` que ya escribieron.

El punto de toda la sesión

El mismo `Solver` que integró el chisme sirve para **cualquier** modelo futuro solo cambiando `f`.

Recapitulando

Modelo del chisme

tres estados discretos, campo suave, un rumor nunca llega al 100%

Clases y herencia

`Solver` agrupa estado y comportamiento; las subclases sobrescriben `step()`

Un patrón, muchos modelos

el mismo código servirá para todo lo que viene en el curso

`mbastidaso@unal.edu.co`